Technical Stack Proposal: Automated Real-Time Virtual Bingo Game

Prepared for: Santiago Villasis (Manager)
Prepared by: Anish Jami, Nathanial Ahluwalia
Document Status: For Review

---

1. Executive Summary

This document proposes a modern, scalable, and enterprise-ready technology stack for building an automated real-time Virtual Bingo Game platform.

The proposed solution replaces the current manual bingo process with a centralized web application that supports automated bingo card generation, real-time gameplay, AI-assisted word calling, automated winner validation, live leaderboard tracking, corporate authentication, and prize notification workflows.

The recommended architecture uses a React/Next.js frontend, a Go-based real-time game backend, a Python FastAPI AI microservice, Azure cloud services, PostgreSQL, Redis, Microsoft Entra ID, GitHub-based CI/CD, and Azure monitoring/security tooling.

The design separates deterministic game logic from AI-assisted features. The Go backend remains the source of truth for game state, winner validation, and auditability, while the AI service provides narration, descriptions, and optional host-assistant functionality.

---

2. Business Context

The current Virtual Bingo process relies on several manual steps, including:

- Generating bingo cards through an external website
- Emailing individual cards to participants
- Hosting gameplay through Microsoft Teams
- Manually calling words or numbers
- Manually typing called words into chat
- Manually validating winners
- Manually tracking 1st, 2nd, and 3rd place
- Manually sending prize notifications or gift cards

This creates operational inefficiency, increased host workload, potential fairness issues, delayed winner recognition, and limited auditability.

The proposed system addresses these limitations by creating a centralized, automated, and auditable application for managing the full bingo game lifecycle.

---

3. Proposed Solution Overview

The Virtual Bingo Game platform will consist of the following major components:

1. **Web frontend** for players, hosts, and administrators
2. **Real-time Go backend** for game state management and WebSocket communication
3. **AI agent microservice** for voice narration, word descriptions, and game-assistant features
4. **PostgreSQL database** for persistent game, user, card, and audit data
5. **Redis** for real-time pub/sub, service communication, and scalable event handling
6. **Azure Blob Storage** for generated assets such as audio, PDFs, or exported result files
7. **Microsoft Entra ID** for corporate single sign-on
8. **Email service** for winner notifications and prize workflows
9. **GitHub and GitHub Actions** for source control and CI/CD
10. **Azure monitoring, logging, and security services** for production operations

---

4. Recommended Technology Stack

Category	Recommended Technology	Purpose
Frontend	React / Next.js	Player, host, and admin web interface
Frontend Hosting	Azure Static Web Apps	Hosting the web application
Backend Language	Go	Core game engine and real-time state management
Real-Time Protocol	Gorilla WebSockets	Live gameplay updates between server and clients
Backend Hosting	Azure Container Apps	Hosting containerized backend services
AI Microservice	Python / FastAPI	AI caller, narration, and host-assistant functionality
AI Integrations	OpenAI API or Gemini API	LLM-generated descriptions and AI responses
Voice / TTS	Azure AI Speech or ElevenLabs	AI voice calling and narration
Database	Azure Database for PostgreSQL Flexible Server	Persistent storage for users, games, cards, calls, winners, and audit logs
Cache / Broker	Azure Cache for Redis	Pub/sub events, temporary state, and service communication

Category	Recommended Technology	Purpose
Object Storage	Azure Blob Storage	Generated cards, audio files, exports, and other assets
Authentication	Microsoft Entra ID	Corporate single sign-on
Authorization	Role-Based Access Control	Managing permissions for admins, hosts, players, and viewers
Email	Azure Communication Services or SendGrid	Winner notifications and prize-related email workflows
Source Control	GitHub	Repository management and code review
CI/CD	GitHub Actions	Automated build, test, and deployment pipelines
Containerization	Docker / Docker Buildx	Packaging frontend/backend services for deployment
Container Registry	Azure Container Registry	Storing built container images
Local Development	Docker Compose	Local multi-service development environment
Secrets Management	Azure Key Vault	Secure storage of API keys, credentials, and connection strings
Identity for Services	Managed Identities	Secure Azure service-to-service authentication
Monitoring	Azure Monitor	Platform-level monitoring and alerting
Application Monitoring	Azure Application Insights	Application telemetry and performance tracking
Logging	Azure Log Analytics	Centralized log collection and querying
Infrastructure as Code	Azure Bicep or Terraform	Repeatable cloud infrastructure provisioning
Testing	Playwright/Cypress, Go tests, Pytest, k6	Frontend, backend, AI service, and load testing
API Documentation	OpenAPI / Swagger	API documentation and contract visibility

5. High-Level Architecture

```
User Browser
  |
```

```
  | HTTPS / WebSocket
  v
React / Next.js Frontend
  |
  | REST APIs + WebSocket Events
  v
Go Game Backend
  |      |
  |      | Pub/Sub Events
  |      v
  |    Redis
  |
  | SQL
  v
PostgreSQL

Go Game Backend
  |
  | Internal API/Event Calls
  v
Python FastAPI AI Service
  |
  | LLM + TTS APIs
  v
OpenAI/Gemini + Azure Speech/ElevenLabs

Supporting Services:
- Microsoft Entra ID for authentication
- Azure Blob Storage for generated assets
- Azure Communication Services or SendGrid for email
- Azure Monitor, Application Insights, and Log Analytics for observability
- Azure Key Vault for secrets management
```

---

6. Component Responsibilities

6.1 Frontend Application

The frontend will provide the user interface for players, hosts, and administrators.

Key responsibilities:

- User login through Microsoft Entra ID
- Game lobby and session joining
- Player bingo card display
- Real-time called-word display

- Player marking interface
- Bingo claim button
- AI caller/chat display
- Host controls for starting, pausing, and ending games
- Live leaderboard for 1st, 2nd, and 3rd place winners
- Admin view for session history and audit records

6.2 Go Game Backend

The Go backend will act as the authoritative game engine.

Key responsibilities:

- Create and manage game sessions
- Generate unique bingo cards
- Assign cards to authenticated players
- Manage called words and game progression
- Broadcast real-time updates through WebSockets
- Track player marks
- Validate bingo claims against winning patterns
- Determine winner ranking order
- Maintain audit logs for game activity
- Expose REST APIs for frontend and admin operations

The backend should remain deterministic and auditable. Winner validation and game state decisions should not be delegated to the AI service.

6.3 Python AI Agent Microservice

The AI service will provide optional enhancement features around narration and player interaction.

Key responsibilities:

- Generate descriptions for called words
- Produce AI caller scripts
- Generate text-to-speech narration
- Send AI-generated messages to the game interface
- Assist the host with non-critical gameplay commentary

The AI service should not control winner validation, card state, or final game outcomes.

6.4 PostgreSQL Database

PostgreSQL will store all persistent application data.

Example data entities:

- Users

- Roles
- Game sessions
- Bingo cards
- Called words
- Player marks
- Bingo claims
- Winners
- Prize notification status
- Audit logs

6.5 Redis

Redis will support real-time and scalable communication patterns.

Use cases:

- Pub/sub between Go and Python services
- Temporary game state caching
- WebSocket scaling across multiple backend instances
- Real-time event distribution

Redis is useful for production scalability, but the core game should still persist authoritative records in PostgreSQL.

6.6 Azure Blob Storage

Blob Storage will be used for generated or exported assets.

Potential stored assets:

- Bingo card PDFs or images
- Generated AI caller audio
- Game result exports
- Archived session files

If cards are rendered directly from JSON in the frontend, Blob Storage may only be needed for audio or exports.

---

7. Authentication and Authorization

Authentication should be implemented using Microsoft Entra ID to support corporate single sign-on.

Authorization should use role-based access control.

Role	Capabilities
Admin	Manage users, view all games, access audit logs, configure system settings
Host	Create games, start/stop sessions, manage gameplay, view winners, trigger prize workflows
Player	Join games, view assigned card, mark squares, claim Bingo
Viewer	Observe game progress and leaderboard only

This separation ensures that users only access the functions appropriate to their role.

---

8. DevOps and Deployment Approach

The project should use GitHub for source control and GitHub Actions for CI/CD.

Recommended CI/CD flow:

```
Developer Push / Pull Request
  |
  v
GitHub Actions
  |
  | Run Tests
  | Build Frontend
  | Build Go Backend Container
  | Build Python AI Service Container
  v
Azure Container Registry
  |
  v
Azure Deployments
  |
  | Frontend -> Azure Static Web Apps
  | Backend -> Azure Container Apps
  | AI Service -> Azure Container Apps
```

Recommended repository structure:

```
virtual-bingo/
  frontend/
  backend-go/
  ai-service/
  infrastructure/
  docs/
```

```
docker-compose.yml
README.md
```

---

9. Security Considerations

Security should be built into the architecture from the start.

Recommended controls:

- Microsoft Entra ID for user authentication
- Role-based access control for authorization
- Azure Key Vault for secrets and API keys
- Managed identities for Azure service access
- HTTPS-only communication
- Environment-based configuration
- No secrets committed to GitHub
- Audit logging for game actions and winner validation
- Input validation on all APIs
- Restricted admin and host permissions

Sensitive values stored in Key Vault should include:

- Database credentials
- Redis connection strings
- LLM API keys
- TTS provider credentials
- Email service credentials
- Entra ID application secrets, if required

---

10. Monitoring and Operations

The platform should include observability for production support and debugging.

Recommended monitoring stack:

- Azure Monitor
- Azure Application Insights
- Azure Log Analytics
- Azure Monitor Alerts

Recommended metrics and logs:

- Active game sessions
- WebSocket connection count

- WebSocket disconnects and reconnects
- Backend response latency
- Failed API requests
- AI service errors
- TTS generation failures
- Email delivery failures
- Authentication failures
- Bingo claim events
- Winner validation results
- Container health and restart counts

This is especially important because the application has real-time behavior and prize-related workflows.

---

11. Testing Strategy

The solution should include automated testing across the frontend, backend, AI service, and real-time workflows.

Test Type	Tooling	Purpose
Frontend tests	Playwright or Cypress	Validate user flows and UI behavior
Backend unit tests	Go testing package	Validate card generation, game state, and winner logic
Python service tests	Pytest	Validate AI service endpoints and fallback behavior
API tests	Postman or Bruno	Validate REST API contracts
Load tests	k6	Validate WebSocket and game-session scalability

Key test cases:

- Unique card generation
- Assigned card ownership enforcement
- Duplicate called-word prevention
- Valid bingo pattern detection
- Invalid bingo claim rejection
- Correct 1st, 2nd, and 3rd place ordering
- Player reconnection handling
- WebSocket event delivery
- Email notification triggering
- Audit log creation

---

12. Key Design Recommendations

1. **Keep Go as the source of truth for game logic.**
   Winner validation, game state, and ranking should be deterministic and auditable.

2. **Use AI as an enhancement layer, not a decision-maker.**
   The AI service should support narration, descriptions, and chat-style interaction, but not decide winners.

3. **Build the core game before advanced AI features.**
   Real-time gameplay, card assignment, validation, and leaderboard functionality are the highest-priority features.

4. **Add Redis when scaling or service decoupling requires it.**
   Redis is valuable for production-scale pub/sub and distributed WebSocket coordination.

5. **Use Azure-native services where appropriate.**
   Since the stack is already Azure-oriented, Azure Container Apps, Static Web Apps, Key Vault, Entra ID, and Application Insights provide a cohesive deployment and operations model.

6. **Include audit logging from the beginning.**
   Since prizes are involved, the system should maintain a clear record of card assignments, called words, claims, validations, and winner order.

---

14. Final Summary and Recommendations

The proposed technology stack is appropriate for an enterprise-grade internal Virtual Bingo Game platform.

It provides:

- A modern web-based user experience
- Reliable real-time gameplay
- Deterministic winner validation
- AI-assisted game narration
- Corporate single sign-on
- Automated communications
- Secure secrets management
- CI/CD and deployment automation
- Monitoring and operational visibility
- A scalable path from MVP to production

The recommended implementation approach is to first build the reliable core game engine and real-time user experience, then layer on AI narration, prize automation, and production hardening features.

This phased approach reduces delivery risk while still supporting the full future-state architecture.